



한무연 – 월간 연구 활동 보고

LLM 보안 연구 기초 다지기 + 실전 취약점 분석 + 대외 활동

고려대학교 인공지능사이버보안학과 (SeeCureLab)

학·석사 과정 이현규



CONTENTS

- I. Review of *Attention Is All You Need*
- II. Vulnerability Analysis of Commercial Software
- III. Analysis of the leaked Claude source code
- IV. CTF Achievements



KOREA
UNIVERSITY
SEJONG CAMPUS

Review of *Attention Is All You Need*

등장 배경 및 중요성

■ 기존 시퀀스 모델의 한계: 순차적 계산

🔄 순환 구조 (Recurrent Structure)

RNN 계열 모델(LSTM, GRU)은
이전 시점($t-1$)의 은닉 상태를 현재 시점(t)의
입력으로 사용.

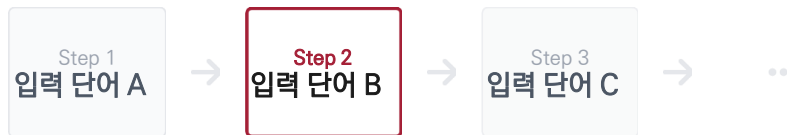
🚫 병렬 처리의 제약

이전 계산이 완료되어야 다음 단계로 넘어갈
수 있어 GPU의 강력한 병렬 처리 성능을 활용
하기 어려움.

⌚ 계산 속도 저하

문장이 길어질수록 비례하여 연산 시간이 기하
급수적으로 늘어나는 병목 현상이 발생.

RNN-BASED SEQUENTIAL PROCESSING



"앞 단어의 처리가 끝날 때까지 기다려야 함."
(Sequential Bottleneck)

등장 배경 및 중요성

■ 기존 시퀀스 모델의 한계: 장거리 의존성

🚫 정보 소실 (Information Loss)

문장이 길어질수록 초기에 입력된 정보가
희미해지거나 소실되는 문제가 발생.

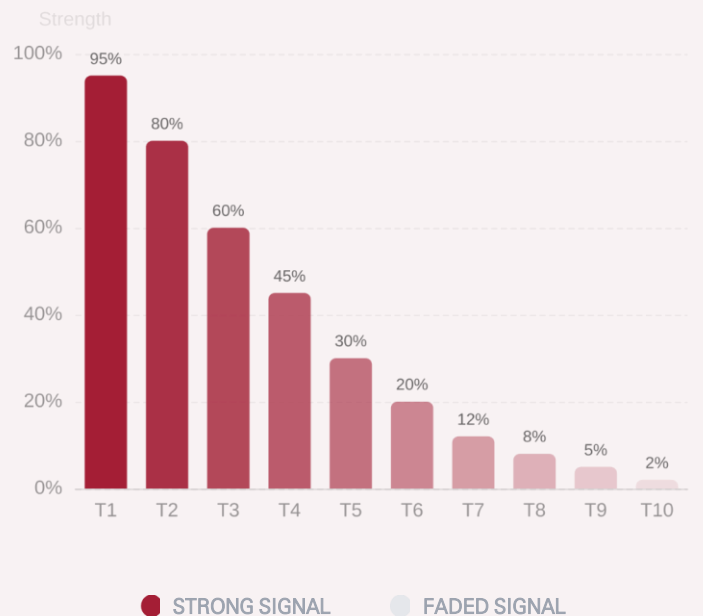
🧠 문맥 파악의 어려움

"그 동물은 길을 건너지 않았다... 왜냐하면 그
것은 너무 피곤했기 때문이다."

→ RNN은 '그것'이 '동물'을 가리킨다는 것을 학습하
기 매우 어려움.

핵심 원인

- Vanishing Gradient (기울기 소실) 현상
- 고정된 크기의 벡터에 모든 정보를 압축하려는 병목 현상



등장 배경 및 중요성

■ Attention 메커니즘의 도입 배경

⚠ 기존 RNN의 한계

- 1 순차적 계산: 긴 문장 처리 시 속도 저하 및 병렬 처리의 어려움
- 2 정보 소실: 초기 입력 정보가 뒤로 갈수록 희미해지는 장기 의존성 문제

"문장이 길어질수록 성능이 급격히 하락하는 병목 현상 발생"

💡 Attention의 역할

"모든 정보를 똑같이 보지 않음"

- ✓ 특정 시점에 관련된 단어에 더 큰 가중치 부여
- ✓ 문맥(Context) 파악 능력의 획기적 향상

EXAMPLE CASE

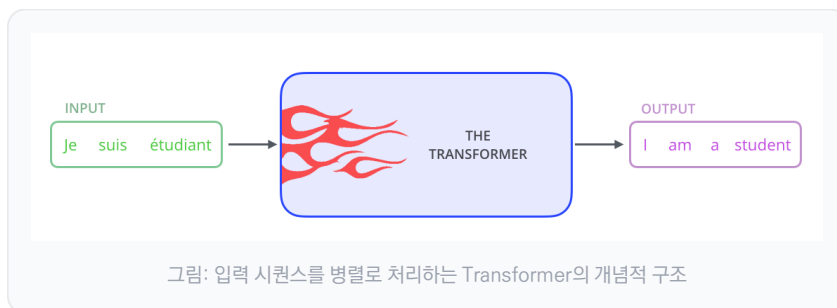
The **animal** didn't cross because **it** was tired.

🔗 Attention mechanism links 'it' to 'animal'

“ ” *Attention은 모델이 입력 시퀀스의 전체를 보되, 가장 유효한 정보에만 집중하게 만드는 돋보기*

Transformer 전체 구조

Transformer의 혁신: 오직 어텐션만으로



순환 및 합성곱 계층 제거

RNN의 순차적 계산과 CNN의 국소적 필터링 없이, 오직 Self-Attention 메커니즘만으로 모델을 구축.

극대화된 병렬 처리 (Parallelization)

입력 시퀀스의 모든 단어를 동시에 처리함으로써, GPU 자원을 최대한 활용하여 학습 시간을 획기적으로 단축.

장거리 의존성 완벽 해결

문장 내의 어떤 위치에 있는 단어라도 단 한 번의 연산으로 직접 연결되어 문맥을 파악.

Transformer 전체 구조

▣ Transformer 모델의 전체 구조 개요

➡ 인코더 (Encoder)

입력 문장(예: 한국어)을 받아 그 의미를 이해하고, **압축된 문맥 정보(Contextualized Representation)**로 변환.

↔ 디코더 (Decoder)

인코더로부터 전달받은 정보를 바탕으로 **목표 언어의 문장(예: 영어)**을 생성. 이전 단어들을 참조하여 다음 단어를 예측.

* Transformer는 이 두 구성 요소를 여러 겹 쌓아(Stack) 복잡한 언어 구조를 학습.

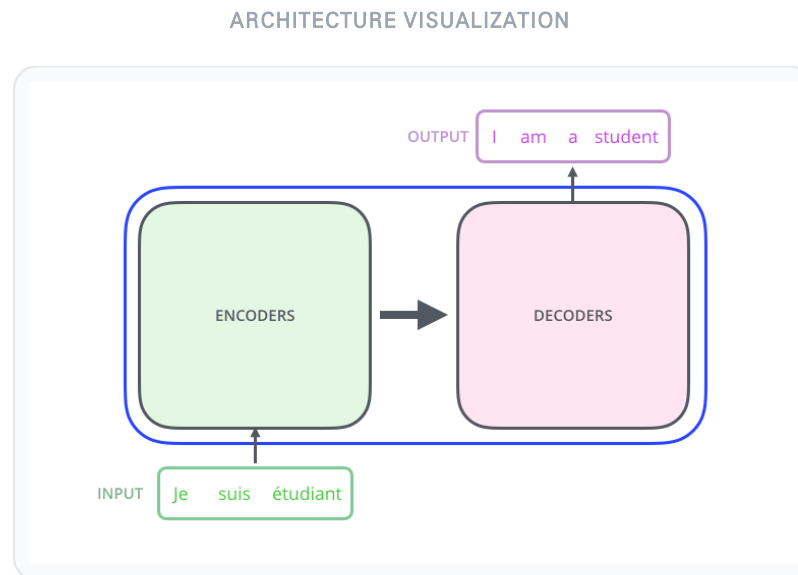
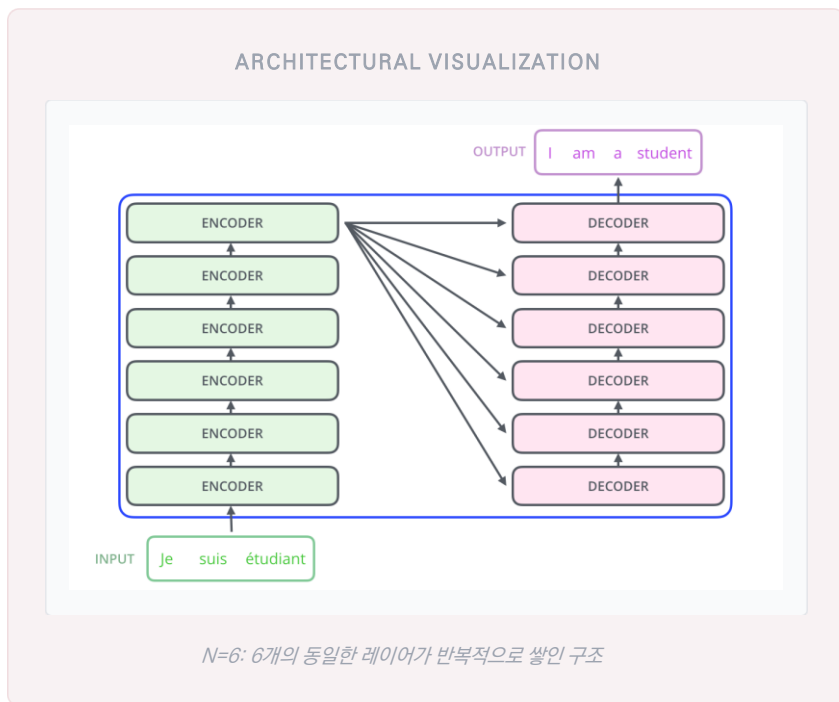


그림: 인코더-디코더 고수준 구조 (출처: The Illustrated Transformer)

Transformer 전체 구조

인코더와 디코더의 스택(stack) 구조



동일한 구조의 반복 (N = 6)

- ✓ 논문에서는 총 6개의 인코더와 6개의 디코더 사용
- ✓ 각 레이어는 가중치를 공유하지 않고 독립적으로 학습

계층적 특징 추출

- ✓ 하위 레이어: 문법적 관계 및 단어 간 직접적 연관성
- ✓ 상위 레이어: 문맥의 고수준 의미와 추상적 정보 파악

일관된 데이터 차원

- ✓ 모든 레이어의 출력 차원은 $d_{model} = 512$ 로 고정
- ✓ 상위 레이어로의 원활한 정보

i 인코더의 최종 출력은 압축된 문맥 정보로서 모든 디코더 레이어로 전달되어 생성 과정을 도움.

Transformer 전체 구조

▣ 인코더의 내부 구성 요소

인코더 레이어를 구성하는 두 가지 핵심 하위 레이어(Sub-layer)



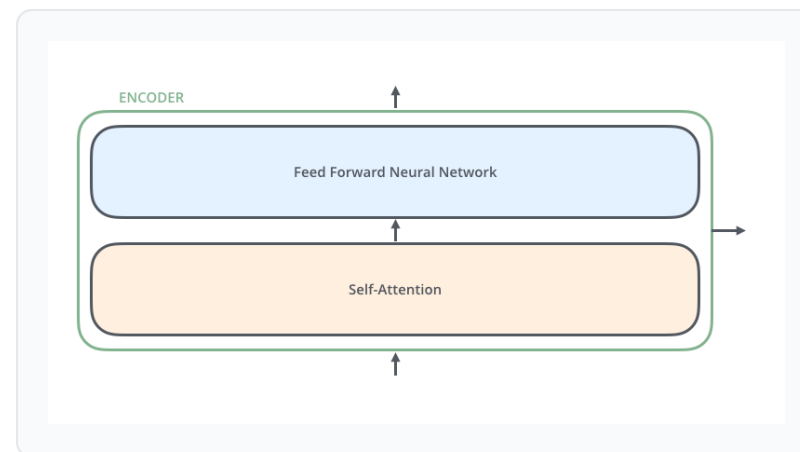
1. Multi-Head Self-Attention

입력 시퀀스 내의 **단어들 간의 관계**를 파악. 특정 단어를 인코딩할 때 문맥적으로 중요한 다른 단어들에 집중하여 풍부한 표현을 생성.



2. Position-wise Feed-Forward

Self-Attention의 출력을 받아 **각 위치별로 독립적으로** 적용되는 완전 연결 신경망. 비선형 변환을 통해 특징(Feature)을 재가공.



[그림] 인코더 내부의 순차적 데이터 흐름

Transformer 전체 구조

▣ 디코더의 구조 – 인코더와 차이점

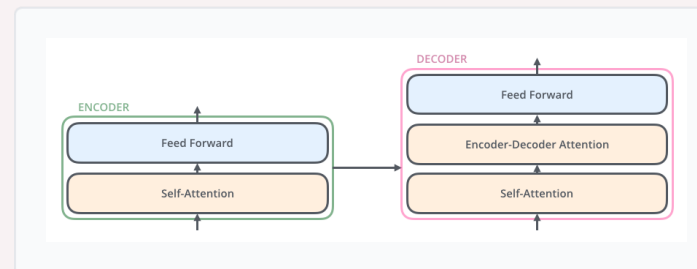
디코더는 인코더와 유사한 구조를 가지지만, **출력 시퀀스 생성**을 위해 두 가지 핵심적인 차이점을 가짐.

🚫 Masked Self-Attention

현재 시점보다 뒤에 있는 단어(미래 정보)를 보지 못하도록 마스킹 처리. 이는 모델이 정답을 미리 보고 예측하는 것을 방지.

🔗 Encoder-Decoder Attention

인코더의 최종 출력을 참고하여, 입력 문장의 어떤 부분에 집중하여 번역어를 생성할지 결정. (인코더와 디코더의 연결 고리)



DECODER LAYER VISUALIZATION

인코더(왼쪽) 대비 디코더(오른쪽)의 추가된 중간 레이어 확인

self-Attention 메커니즘

■ Self-Attention의 이해

문장 내의 특정 단어를 이해하기 위해 다른 모든 단어들과의 관계를 직접적으로 계산하는 방식

왜 Self-Attention이 중요한가?

- ✓ **전역적 문맥 파악:** 문장이 길어져도 멀리 떨어진 단어 간의 관계를 즉각적으로 연결
- ✓ **동적 가중치 할당:** 현재 단어의 의미를 결정짓는 핵심 단어에 더 높은 '집중도(Attention)' 부여

간단한 비유

“여러 사람이 모인 파티장에서 누군가 ‘그분’이라고 말했다를 때, 주변 상황 and 대화 맥락을 살피 그가 누구인지 즉시 알아차리는 과정과 같음.”

self-Attention 메커니즘

▣ Q, K, V: 데이터 처리를 위한 세 가지 핵심 벡터 (한 문장에서)

🔍 Query (질문)

"내가 지금 집중해야 할 단어는무엇인가?"

현재 단어의 탐색 요청

🏷️ Key (색인)

"나는 이런 의미를 가진 단어다 (매칭용 태그)"

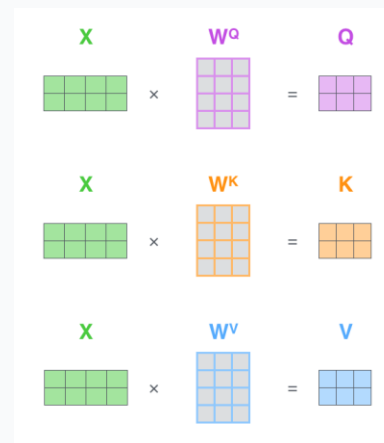
다른 단어와의 관련성 측정 기준

📄 Value (내용)

"내가 가진 실제 의미 정보값"

최종적으로 전달할 실제 데이터

벡터 생성 과정 (MATRIX MULTIPLICATION)



* 각 가중치 행렬(W)은 학습을 통해 최적화되며, $d_k = d_v = 64$ 차원으로 투영.

"각 단어는 질문(Q)을 던지고, 다른 단어들의 색인(K)과 비교하여, 가장 적절한 내용(V)을 가져옴."

self-Attention 메커니즘

Self-Attention: 점수와 스케일링

1 점수(Score) 계산

현재 단어의 **Query(q)**와 문장 내 모든 단어의 **Key(k)** 간의 연관성을 측정.

$$\text{Score} = q \cdot k$$

* 내적(Dot Product) 값이 높을수록 두 단어는 서로 밀접한 관계임을 의미

2 스케일링(Scaling)

내적 결과값을 $\sqrt{d_k}$ 로 나누어 줌. (논문 기준 $d_k=64$ 이므로 8로 스케일링)

$$\text{Scaled Score} = \text{Score} / \sqrt{d_k}$$

i Why? 점수가 너무 커지면 Softmax 함수에서 기울기(Gradient)가 소실되어 학습이 불안정해지는 것을 방지.

CALCULATION VISUALIZATION

Input	Thinking	Machines
Embedding	x_1	x_2
Queries	q_1	q_2
Keys	k_1	k_2
Values	v_1	v_2
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12

THINKING SCORE

112 → 14

MACHINES SCORE

96 → 12

"스케일링은 단순히 값을 줄이는 것이 아니라, Softmax의 입력 범위를 적절히 유지하여 효과적인 역전파(Backpropagation)를 가능하게 함."

self-Attention 메커니즘

Self-Attention 계산 단계: 가중합과 최종 출력

3 Softmax 적용 (정규화)

스케일링된 점수를 0과 1 사이의 확률로 변환. 모든 단어에 대한 가중치의 합은 1이 되며, 이는 모델이 어떤 단어에 **얼마나 집중할지**를 수치화한 것임.

4 가중합 (Weighted Sum)

각 단어의 가중치를 해당 단어의 **Value (V) 벡터**에 곱함. 관련성이 높은 단어의 정보는 크게 반영되고, 낮은 단어는 희석됨.

✓ 최종 출력 (Z)

가중치가 적용된 Value 벡터들을 모두 합산하여 현재 단어의 **문맥적 의미**가 포함된 최종 벡터 z 를 생성.

CALCULATION PROCESS VISUALIZATION

Input	Thinking	Machines
Embedding	x_1	x_2
Queries	q_1	q_2
Keys	k_1	k_2
Values	v_1	v_2
Score	$q_1 \cdot k_1 = 112$	$q_2 \cdot k_2 = 96$
Divide by $8 (\sqrt{d_k})$	14	12
Softmax	0.88	0.12

SOFTMAX RESULT (THINKING)

0.88 (88%)

SOFTMAX RESULT (MACHINES)

0.12 (12%)

* 위 예시에서 'Thinking'은 자기 자신과 88%의 높은 관련성을 보이며 정보를 통합.

self-Attention 메커니즘

행렬 연산을 통한 효율적 병렬 처리

단어 벡터에서 문장 행렬(X)로

- ✓ 입력 문장의 단어 임베딩을 쌓아 행렬 **X** 생성
- ✓ 학습 가능한 가중치 행렬 **WQ, WK, WV** 한 번에 곱

⚡ 왜 행렬 연산인가?

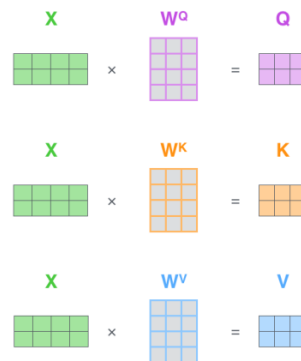
병렬 처리

GPU 코어 활용 극대화

속도 향상

단일 연산으로 전체 처리

MATRIX GENERATION LOGIC



Q, K, V 행렬 생성 병렬 행렬 곱셈 과정

self-Attention 메커니즘

■ Multi-Head Attention: Transformer은 Self-Attention을 한 번만 하는 게 아니라 여러 번 수행

? 왜 여러 개의 '헤드'가 필요할까요?

단일 어텐션은 문장 내의 **한 가지 종류의 관계**만 포착하기 쉬움. Multi-Head Attention은 모델이 서로 다른 위치에 있는 정보를 다양한 관점에서 동시에 처리하도록 도움.

🔧 핵심 작동 원리

- ✔ **병렬 처리**: 8개의 독립적인 어텐션 헤드가 병렬로 계산.
- ✔ **독립적 가중치**: 각 헤드는 고유한 가중치 행렬(W_Q, W_K, W_V)을 학습.
- ✔ **표현 공간 확장**: 문법적 관계, 의미적 관계 등을 서로 다른 헤드가 분담하여 포착.

"마치 8명의 전문가가 각자의 전문 분야(문법, 문맥, 대명사 등)에 집중하여 문장을 분석하는 것과 같음."

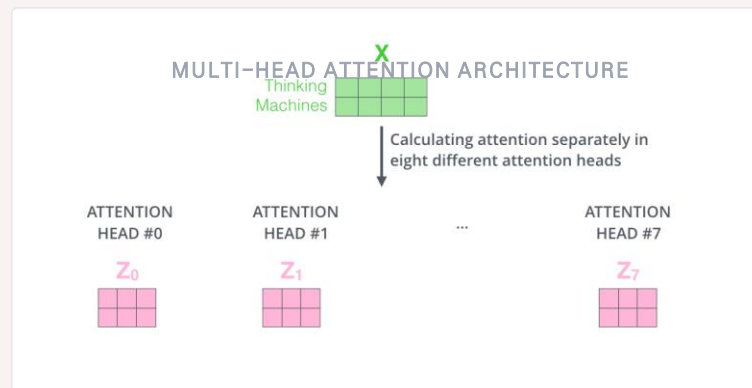


FIG. MULTI-HEADED SELF-ATTENTION PROCESS (H=8)

self-Attention 메커니즘

Multi-Head Attention의 결과 통합

✖ 통합 과정 (Concatenation)

- 1 각 Attention Head($Z_0, Z_1, \dots, Z_7$)가 생성한 서로 다른 문맥 정보를 가로로 이어 붙임.
- 2 이어 붙인 큰 행렬에 학습 가능한 가중치 행렬 W^O 를 곱하는 선형 변환을 수행.
- 3 최종적으로 하나의 Z 행렬로 압축되어 다음 레이어(Feed-Forward)로 전달.

왜 통합이 필요한가?

각 헤드는 문법적 관계, 의미론적 관계 등 서로 다른 표현 부분 공간(Subspaces) 정보를 포착. 이를 하나로 합침으로써 모델은 문장의 다각적인 의미를 동시에 이해할 수 있게 됨.

INTEGRATION VISUALIZATION

1) Concatenate all the attention heads

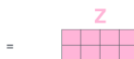


2) Multiply with a weight matrix W^O that was trained jointly with the model

x



3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



[그림] Multi-Head Attention의 결과를 병합하고 가중치 행렬 W^O 와 곱하는 과정

상세 작동 방식 및 학습

▣ Positional Encoding: 위치 정보의 주입

⚠ 순서 정보의 부재

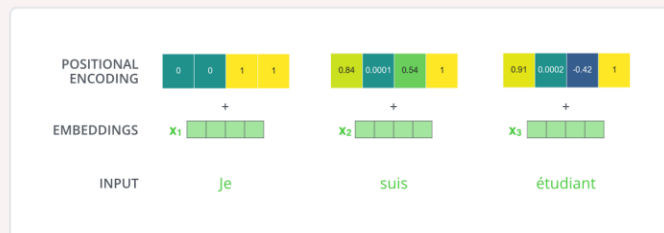
Transformer는 RNN처럼 순차적으로 입력을 처리하지 않고 모든 단어를 **동시에 병렬 처리**. 따라서 단어의 위치(순서)를 인지할 수 없는 구조적 한계가 있음.

+ 해결책: 위치 정보 더하기

각 단어의 **Embedding** 벡터에 위치 정보를 담은 고유한 **Positional Encoding** 벡터를 더해줌으로써, 모델이 문장 내 단어의 상대적/절대적 위치를 파악하게 함.

"단어의 의미뿐만 아니라, 그 단어가 문장의 어디에 위치하는지에 대한 '좌표'를 함께 제공하는 것과 같음."

CONCEPTUAL DIAGRAM



INPUT EMBEDDING

+

POSITIONAL ENCODING

상세 작동 방식 및 학습

▣ 디코더의 마스킹 Self-Attention

🔒 마스킹(Masking)이란?

현재 시점의 단어를 생성할 때, 미래 시점의 단어 정보를 참조하지 못하도록 차단하는 기법.

왜 필요한가? (Auto-regressive)

학습 시에는 정답 문장이 한꺼번에 입력되지만, 실제 추론 시에는 단어를 하나씩 순차적으로 생성해야 함. **정답 유출(Cheating)**을 방지하여 실전과 같은 환경을 조성.

핵심 요약

"모델이 과거의 단어들만 보고 다음 단어를 예측하도록 훈련."

ATTENTION MASK MATRIX

V1	🔒	🔒	🔒
V1	V2	🔒	🔒
V1	V2	V3	🔒
V1	V2	V3	V4
W1	W2	W3	W4

* 위쪽 삼각형(Masked area)은 학습 시 Softmax 이전에 $-\infty$ 로 처리되어 가중치가 0이 됨.

상세 작동 방식 및 학습

▣ 최종 출력층: 단어 예측 과정

1 Linear Layer

최상위 디코더의 출력 벡터를 모델의 전체 단어장 크기(*vocab_size*)인 로짓(logits) 벡터로 투영.

Output Vocabulary						
WORD	a	am	i	thanks	student	<eos>
INDEX	0	1	2	3	4	5

2 Softmax Layer

각 로짓 값을 0~1 사이의 확률값으로 변환하며, 모든 단어 확률의 합은 **1.0**



3 Word Selection

가장 높은 확률을 가진 단어를 선택.

- ✓ Greedy: 가장 높은 확률 선택
- ✓ Beam Search: 유망 후보군 탐색

i Auto-regressive 특성

이 과정은 **<eos>** (End of Sentence) 토큰이 생성될 때까지 반복. 선택된 단어는 다시 다음 스텝의 디코더 입력으로 사용.

실험 결과 및 성능 분석

■ 기계 번역 성능 분석 (WMT 2014)

Transformer 모델이 달성한 성능 지표.

🏆 새로운 SOTA 달성

Transformer (big) 모델은 기존의 모든 앙상블 모델을 능가하는 최고 성능을 단일 모델로 기록

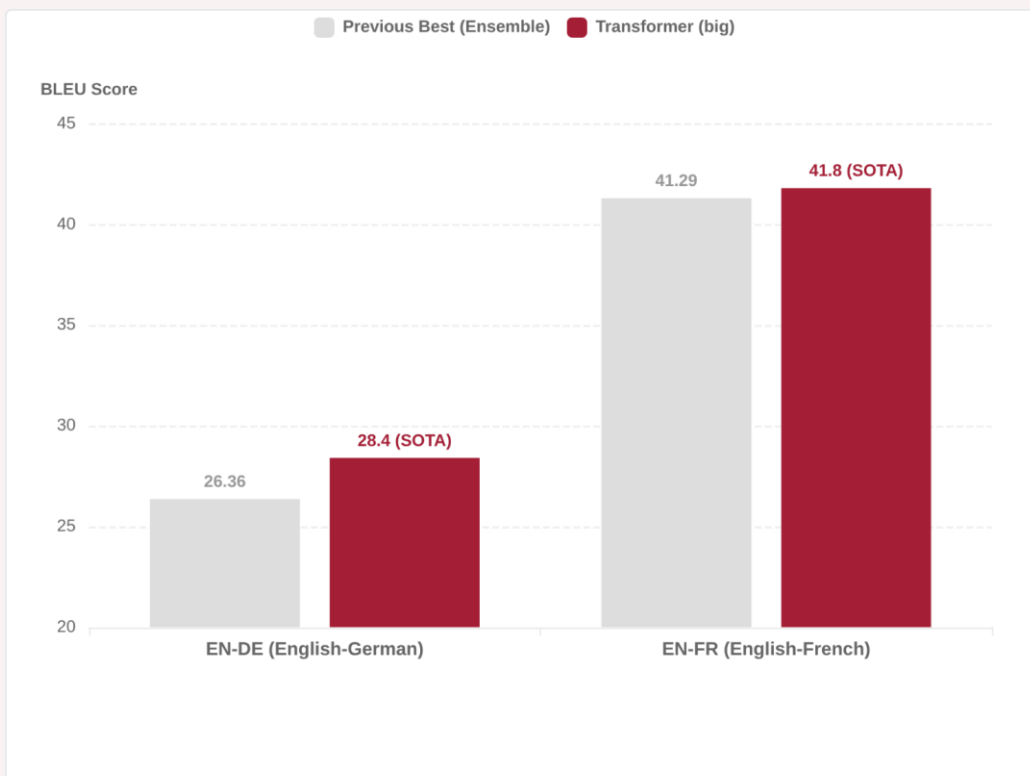
📈 EN-DE 성능 비약적 향상

기존 최고 기록(26.36) 대비 **+2.0 BLEU** 이상의 압도적인 성능 향상을 보여줌.

CONCLUSION

"Attention만으로도 기존의 RNN/CNN 기반 앙상블 모델보다 뛰어난 성능 구현이 가능함을 입증"

BLEU SCORE COMPARISON (WMT 2014)



비교 대상: GNMT, CONVS2S, MOE 등 기존 주요 아키텍처 (앙상블 포함)

실험 결과 및 성능 분석

▣ 효율성 분석: 학습 비용과 속도

압도적인 학습 속도

병렬 처리가 불가능한 RNN 기반 모델과 달리, Transformer는 모든 시퀀스를 동시에 처리하여 학습 시간을 획기적으로 단축함.

낮은 학습 비용 (FLOPs)

기존 SOTA 모델인 GNMT나 ConvS2S 양상블 모델 대비 수십 배 적은 연산량으로 더 높은 성능을 달성.

KEY TAKEAWAY

"더 적은 자원으로 더 짧은 시간에 더 좋은 모델을 만들 수 있는 경제적 우위를 확보."

결론 및 향후 연구 방향

■ 결론: Transformer의 의의와 기여



구조적 혁신

순환(Recurrence)이나 합성곱(Convolution) 없이 오직 **Attention**만으로 시퀀스 데이터를 처리하는 새로운 패러다임을 제시.



압도적 성능

기존 SOTA 모델들을 큰 차이로 앞지르며 기계 번역 분야의 새로운 기준을 세웠고, 학습 효율성 또한 극대화함.



무한한 확장성

번역을 넘어 이미지, 오디오, 비디오 등 모든 데이터를 처리할 수 있는 **범용 아키텍처**로서의 가능성을 엿.

결론 및 향후 연구 방향

▣ 향후 연구 방향과 확장성



멀티모달 확장

텍스트를 넘어 이미지, 오디오, 비디오 등 다양한 형태의 입출력을 처리하는 연구로 확장.

Image

Audio

Video



계산 효율성 개선

긴 시퀀스 처리 시 발생하는 $O(n^2)$ 복잡도 문제를 해결하기 위해 국소적/제한적 어텐션 기법 연구.

Local Attention

Sparse Attention



비순차적 생성

디코더의 Auto-regressive 특성을 개선하여 생성 속도를 높이고 병렬 생성을 가능케 하는 연구.

Parallel Gen

Speed-up

왜 해당 논문을 읽었나?

▣ 대부분의 보안 이슈도 이 구조 위에서 발생

정리

- 공격 표면을 이해하려면 모델의 기본 동작 원리를 알아야 함 - 입력을 그냥 문장으로 처리하는게 아닌 토큰 간 관계를 attention으로 계산
- 작은 문구 하나가 모델의 행동을 바꿀 수 있음 - transformer는 모든 토큰이 서로 참조할 수 있기 때문
- 입력 필터링, prompt isolation, context partitioning, attention steering, activation monitoring 같은 방어는 전부 모델이 어떤 방식으로 정보 결합하는지에 따라 설계 가능

LLM 보안은 출력만 막는 문제가 아니라, Transformer가 정보를 결합하는 방식을 통제하는 문제라는 것을 알 수 있었음.



KOREA
UNIVERSITY
SEJONG CAMPUS

Vulnerability Analysis of Commercial Software

Vulnerability Analysis of Commercial Software

▣ 분석 대상: LLM 기반 AI 애플리케이션 시각화 및 운영 툴 (오픈소스)

취약점 정보

- BAC 취약점 발견 – 서버 측 권한 검증 로직 누락
- 영향 범위: 팀 전체 리소스 (낮은 권한 사용자의 무단 쓰기 가능)
- 보안 권고 사항
 - 백엔드 권한 검증 강화
 - 다중 방어 계층 구성
- 잠재적 영향력
 - 서비스 무결성 훼손 위험
 - 가용성 영향 가능성
 - 내부 위협 발생



KOREA
UNIVERSITY
SEJONG CAMPUS

Analysis of the leaked Claude source code

Analysis of the leaked Claude source code

■ When did it happen?

유출 경과

- 2026년 3월 31일, Claude Code npm 배포본에 source map이 포함되며 내부 코드가 외부에 노출
- Anthropic은 이를 해킹이 아닌 배포 실수라고 설명

Mar 31, 2026 · Business

Anthropic leaked 500,000 lines of its own source code

 Madison Mills

    Add Axios on Google



Illustration: Sarah Green/Arise

Source material powering Anthropic's Claude Code leaked for the second time in just over a year, publicly exposing the AI coding tool's full architecture, unreleased features and internal model performance data.

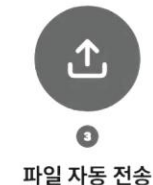
Analysis of the leaked Claude source code

▣ What was analyzed?

유출 내용

- 공개된 코드는 약 50만 줄, 1900개 안팎 파일 규모로 빠르게 확산
- 해당 유출된 코드에는 에이전트 하네스, 워크플로, 미공개 기능(약 40개 이상), 내부 실험 흔적이 발견됨

KAIROS 자율 동작 프로세스



사용자가 아무것도 안 시켜도 알아서 돌아가는 AI

Analysis of the leaked Claude source code

■ What can be done with it?

알 수 있는 것

- 기존 모델 같은 경우 Claude로 로그인하거나 Claude API를 사용해야 하는데 이를 Ollama를 통해 무료로 동작시킴 (이미 오픈소스로 공개됨)
- 이번 유출로 에이전트 하니스와 정책 레이어가 드러나면서, 구조 분석과 오남용 시도 가능성이 커짐
- 이를 통해 단순 API보다 하네스 설계와 운영 구조가 더 큰 경쟁력임을 보여줌
- Ex) 하드 프롬프팅된 요소를 제거하고 모든 동작을 허용하도록 패치하여 솔루션 판매



KOREA
UNIVERSITY
SEJONG CAMPUS

CTF Achievements

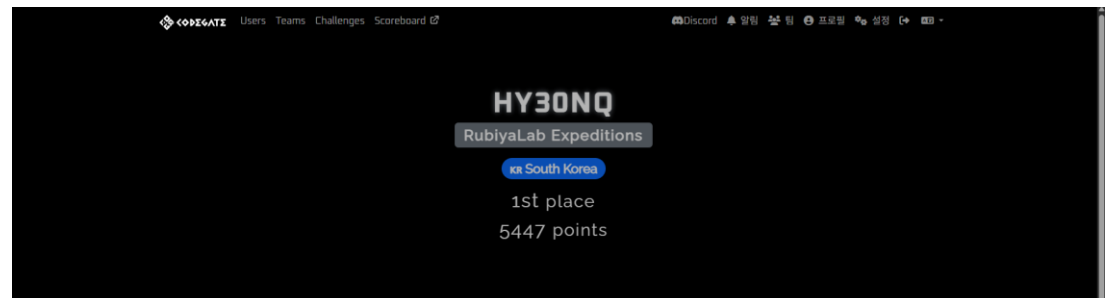
CTF Achievements

CODEGATE 2026

진행 내용

- 팀에서 리버싱 용 CTF Agent 개발 참여 (기존 CTF 라이트업 바탕으로 학습 + MCP 툴 연결)
- 844개 팀 중 일반부 예선 1등 → 본선 진출

CODEGATE 2026			Qualifiers		
No.	일반(General)		주니어(Junior)		
25th Winner	Blue Water	-	1	pandas	6521
1	RubiyaLab Expeditions	5447	2	c-bass	6521
2	BunkyoWesterns	5059	3	==I LOVE SILVERWOLF! == =我最爱银狼姐姐了捏!!	6521
3	r3kapig	5059	4	zuhwand	6223
4	육개장치즈라면	4638	5	tiexion	5602
5	TeamH4C	4638	6	nullbyte	5602
6	GMO Ierae	4638	7	김필립	5602
7	AAA	4638	8	sh1nY	5258
8	Infobahn	4638	9	Yeonba	5258
9	Sweep Duck	4638	10	tpwnd	5258
10	검해출신취업준비생	4559	11	MOFW	5124
11	GYG	4388	12	alsgn1_	5124
12	따마	4388	13	공법학 학생	4870
13	MINGA	4388	14	0n3n3ss	4870
14	blue-lotus	4388	15	Spaghet	4870
15	UIT-Refund	4388	16	송원호	4780
16	H-T8	4309	17	T00th1ess	4780
17	The Amazing Digital Orange!	4250	18	itsk/miot	4392
18	F1ux	4250	19	41one	4392
19	김용송팬클럽	4250	20	comet	4392



감사합니다.
